# Experimentation Fundamentals 🧪

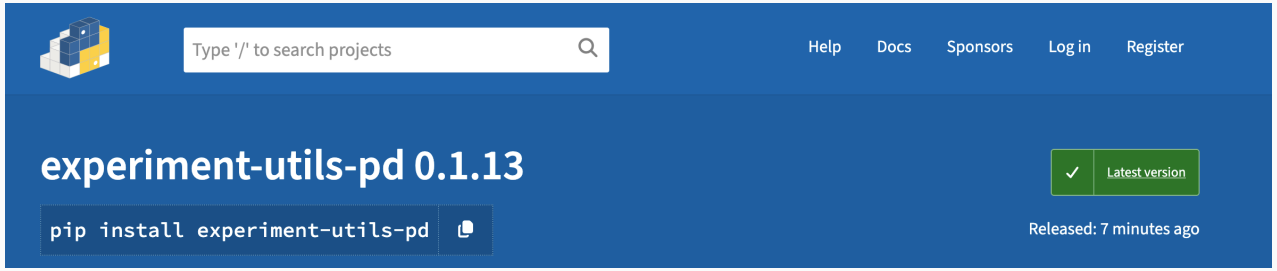
Brief introduction and intuitions

Sebastian Daza

# Outline 🗺️

- Causal Inference Fundamentals
- Experimental Design
  - Power Analysis & Sample Size
  - Sampling & Stratification
- Analysis
  - Non-Compliance & Confounding

# Python Package: `experiment-utils-pd` 📦




Key components:

- `ExperimentAnalyzer` - A/B test analysis, IV, IPW, regression adjustment, retrodesign
- `PowerSim` - Sample size & power calculations, retrodesign
- `utils` - Balanced random assignment

**Links:** PyPI | GitHub

# Causal Inference Fundamentals

# The Ladder of Causation (Pearl) 📊

| Rung | Level | Question | DS Role |
|---|---|---|---|
| 1 | Association *(Owl)* | *"What is?"* | Prediction |
| 2 | Intervention *(Baby)* | *"What if I do...?"* | Decisions |
| 3 | Counterfactual *(Scientist)* | *"What if I had...?"* | ITE / CATE |

- Rung 1: $P(Y \mid X)$ — patterns in observed data; home of predictive ML
- Rung 2: $P(Y \mid do(X))$ — effect of actions; needs experiments or assumptions
- Rung 3: $P(Y_x \mid x', y')$ — individual counterfactuals; structural causal models

# Rung 1 — The Owl: Seeing Patterns 🦉

$P(Y \mid X)$ — *"Given I observe X, what is Y likely to be?"*

- **Industry home:** risk prediction, churn, fraud detection, recommendations
- **Task:** find patterns in data → guess an outcome
- **Powerful for:** monitoring, alerting, targeting, ranking
- **Cannot tell you:** what to *change* to get a different outcome

A churn model tells you who will churn — not what intervention would prevent it

# The Data Validity Cliff ⚠️

Your data has solid validity for prediction — then you step off a cliff the moment you use it to justify an intervention.

The cliff is confounding: $P(Y \mid X) \neq P(Y \mid do(X))$ — your model learned *who* tends to have high $Y$, not *what causes* $Y$ to change

## Selection vs. Policy

- **Observation:** *"Users who adopt Feature X have 40% higher engagement"*
- **Decision:** Ship Feature X to everyone → engagement will rise
- **Reality:** Engaged users self-selected into Feature X — the engagement was already there

# Rung 2 — The Baby: Doing & Deciding 🧪

$P(Y \mid do(X))$ — *"What happens if we intervene?"*
Business decisions live here

- A/B test: randomization manufactures the *do* — cleanest path to Rung 2
- **Not always possible:** ethical constraints, time horizons, cost, irreversibility

**When you can't experiment → Identification:**

- Build a bridge: $P(Y \mid X) \to P(Y \mid do(X))$ using assumptions
- Tools: DiD, IV, IPW, regression discontinuity, synthetic control
- The data can never give you the assumptions — they come from theory

# Rung 3 — The Scientist: Imagining 🔬

**A/B tests → ATE**: average effect across the population

Nothing specific about individuals or subgroups

**Rung 3 asks:** *"What would have happened to this unit had we acted differently?"*

Requires a structural causal model — logic + domain knowledge, not just data

**Email campaign example:**

- Rung 2: "Does the campaign increase purchases on average?" → holdout test → ATE
- Rung 3: "Would *this* customer have bought without the email?" → purchase intent model

# The Fundamental Problem (Rung 2) 🧪

For any unit, we only observe one potential outcome — never both

- Dr. X got the new onboarding flow → 8 bookings/week
- We cannot observe: same doctor with the old flow → **?**
- Individual causal effects are unobservable

**Rung 2 solution**: compare groups → estimate the ATE

- A/B test: random assignment creates a counterfactual group

# Key Assumptions for Valid Experiments 📝

1. **Independence (Randomization)**
   - Random treatment assignment creates exchangeable groups
2. **Stable Unit Treatment Value Assumption (SUTVA)**
   - No interference: Units don't affect each other
   - Consistency: Treatment uniformly defined
   - Violations: Network effects, marketplace spillovers
3. **Compliance**
   - Treatment received matches treatment assigned

# Internal vs External Validity 🎯

Internal Validity: Can we trust the causal inference within our experiment?

- **Threats:** Selection bias, implementation errors (SRM), measurement errors, SUTVA violations

External Validity: Can we generalize beyond our experiment?

- **Threats:** Non-representative samples, novelty effects, seasonal/context factors

# Experimental Designs: How to Assign Treatments? 🧪

## Between-Subjects Design:

- Each unit receives only one condition; compare different units (A vs B)
- *E.g.*, 50% see version A, 50% see version B
- **Limitation:** More participants needed, lower power

## Within-Subjects Design:

- Each unit receives all conditions at different times; compare same unit
- *E.g.*, All users see A in week 1, B in week 2
- **Limitation:** Order/carryover effects; not for irreversible treatments

# Power Analysis & Sample Size

# Power 💪

**Power** = Probability of detecting an effect when it exists

## Definition

- $\alpha$ (Type I error): False positive rate, typically 0.05
- $\beta$ (Type II error): False negative rate, typically 0.20
- Power = $1 - \beta$, typically 0.80 (80%)

## Input we need to calculate power:

- **Expected effect size** : Magnitude of difference
- **Variance**: Variability of the outcome metric
- **Significance level**: Threshold for statistical significance
- **Sample size**: Number of units per group

For more details, look at: blog's post on power analysis

# Minimum Detectable Effect (MDE) 🔎

**MDE** = Smallest effect size reliably detected at 80% power, $\alpha$ = 0.05, and sample $n$.

## How to define MDE:

1. **Historical:** Based on past experiments and benchmarks (typically 1-5%)
2. **Resource-constrained:** Achievable with sample (e.g., 50K users $\rightarrow$ 3% MDE)

## If MDE is unreasonable & sample is limited:

- Avoid running under-powered experiments, risk of winner's curse (exaggerated estimates)
- Use quasi-experimental methods instead!

# Small Effects + Limited Traffic: What to Do? 💡

1. **Reduce variance**: CUPED (pre-experiment covariate adjustment), regression adjustment (control for pre-treatment covariates), stratified randomization
2. **Use sensitive metrics**: surrogate/proxy metrics
3. **Redesign experiment**: within-subjects, responsive subpopulations, longer duration
4. **Accumulate evidence**: Meta-analysis (pooled experiments)
5. **Go quasi-experimental**: DiD, IPW, Synthetic Control, Geo-experiments, Interrupted Time Series

# Power Analysis in Practice 🔍

1. **Always explore multiple scenarios** — not just one power calculation
   - Jointly optimize: 80% power + realistic MDE + acceptable duration
   - In 1–2% MDE territory: proxy metrics, CUPED
2. **MDE is the most constrained parameter** — rarely inflated to compensate
   - Should reflect minimum business-relevant effect, not statistical convenience
   - If MDE is unrealistic, the experiment probably shouldn't run (quasi-exp instead)
3. **Heuristics as smart defaults**
   - Keep power $\geq$ 80% (most respected threshold)
   - Cap MDE at 5%; two-tailed tests preferred unless strong prior

# Multiple Comparisons Problem

**Problem:** Every test has a 5% false positive rate by chance. Testing $m$ metrics/variants simultaneously inflates this:

$$P(\text{at least 1 false positive}) = 1 - (1 - \alpha)^m \quad \xrightarrow{m=10} \quad \approx 40\%$$

**Two correction strategies:**

**FWER** Control probability of *any* false positive
Bonferroni, Holm-Bonferroni — stricter, fewer discoveries

**FDR** Control the *proportion* of false positives among significant results
Benjamini-Hochberg — less strict, better when testing many metrics

# Find Sample Size & Power 👩‍💻

`PowerSim` uses a simulation approach, more useful for complex designs and metrics (compliance, multiple variants, etc.)

```
from experiment_utils import PowerSim

p = PowerSim(metric='proportion',
             relative_effect=False,
             variants=2,
             nsim=1000,
             alpha=0.05,
             alternative='two-tailed',
             comparisons=[(1, 0), (2, 0), (2, 1)],
             correction='holm')
```

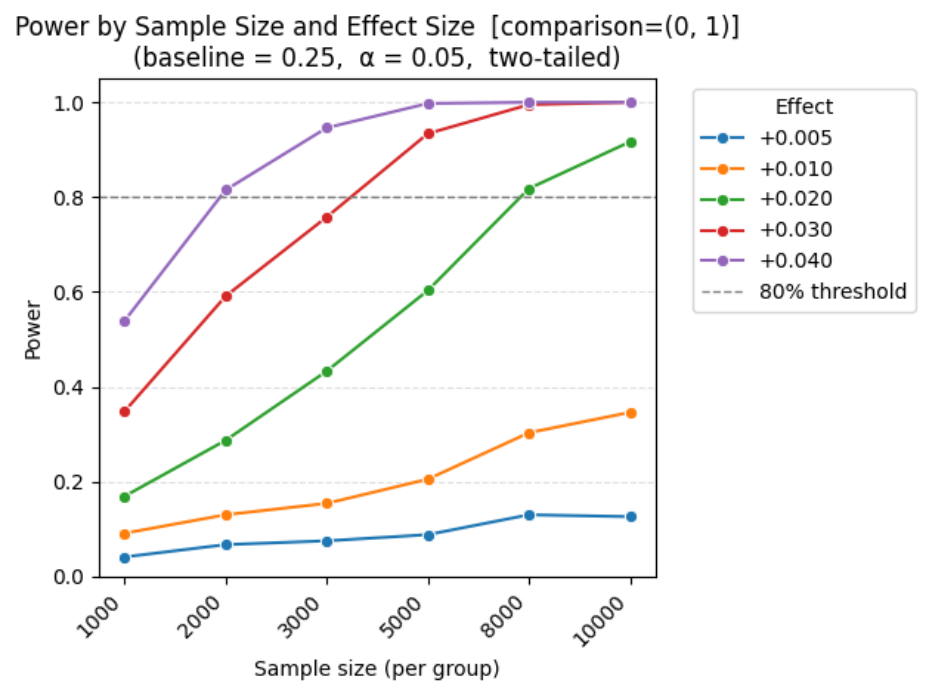
# Find Sample Size & Power 👩‍💻

```
result = p.find_sample_size(
    power=0.80,
    baseline=0.10,
    effect=[0.03, 0.05],
    # compliance=0.80,
    optimize_allocation=True,
    tolerance=0.0001)
)
```

```
Using sample size 10778 (driven by (1, 2))
Optimized sample sizes: {'control': 1167, 'variant_1': 4805, 'variant_2': 4805}
Achieved power: {'(0, 1)': 0.815, '(0, 2)': 0.994, '(1, 2)': 0.808}
```

# Exploring the Power Surface 📈

```
p = PowerSim(metric='proportion',
             relative_effect=False,
             variants=1,
             nsim=1000)

rr = p.grid_sim_power(baseline_rates=0.25,
                      effects=[0.005, 0.01, 0.02, 0.03, 0.04],
                      sample_sizes=[1000, 2000, 3000, 5000, 8000, 10000],
                      color_by='effect',
                      threads=16,
                      plot=True)
```



# Sampling & Covariate Balance

# Sampling Methods for Experiments ✂️

**Why sampling matters:**

- Limited resources (budget, time, traffic)
- Reduces variance - Better precision in estimates
- Increases credibility - Shows randomization worked properly

**Two main approaches:**

- Random Sampling - Simple random selection
  - Easy to implement; may result in imbalanced small segments
- Stratified Sampling - Sample within dimensions
  - Ensures representation across strata, better for heterogeneous populations
  - Can use proportional or equal allocation

# Blocking / Stratified Sampling 👩‍💻

```python
from experiment_utils import balanced_random_assignment

# random allocation
treatment_unblock = balanced_random_assignment(
    df,
    variants=['treatment', 'control'],
    allocation_ratio=1/2,
    balance_covariates=['age', 'previous_purchases', 'days_since_signup'],
    seed=4321
)
```

```
Balance Check After Assignment

Comparison: control (n=1,500) vs treatment (n=1,500)

         covariate  n_control  n_treatment  mean_control  mean_treatment      smd balanced
               age       1500         1500     39.629457       39.578661 0.005167        y
previous_purchases       1500         1500      3.072000        2.958000 0.065706        y
 days_since_signup       1500         1500    367.491830      345.775354 0.060344        y

Summary: 3/3 covariates balanced (|SMD| < 0.1)
Mean |SMD|: 0.0437
Max |SMD|: 0.0657
```

# Blocking / Stratified Sampling 👩‍💻

```
# stratified allocation
treatment_block = balanced_random_assignment(
    df,
    variants=['treatment', 'control'],
    allocation_ratio=1/2,
    stratification_covariates=['age', 'previous_purchases', 'days_since_signup'],
    seed=4321
)
```

```
Balance Check After Assignment

Comparison: treatment (n=1,500) vs control (n=1,500)
         covariate  n_treatment  n_control  mean_treatment  mean_control      smd balanced
               age         1500       1500       39.656748     39.551370 0.010718        y
previous_purchases         1500       1500        3.015333      3.014667 0.000384        y
 days_since_signup         1500       1500      358.103009    355.164175 0.008163        y

Summary: 3/3 covariates balanced (|SMD| < 0.1)
Mean |SMD|: 0.0064
Max |SMD|: 0.0107
```

# Analysis

# Components 🔪

Key components:

- Primary metrics
  - Pre-specified metrics (e.g., revenue, conversion)
  - Multiple tests?
- Guardrail metrics
  - Safety checks (e.g., load time, error rates)
  - Implementation metrics
- Sample Ratio Mismatch (SRM)
  - Does split match expectation?
  - SRM signals implementation issues or bias

# Distribution Assumptions: Don't Overthink It 🧠

**Why not useful:**

- CLT (Central Limit Theorem): Sample means ≈ normal for large n (>30-50), regardless of distribution
- Normality tests are too sensitive; sampling distribution matters, not population

**What to do:**

- **Large samples:** Use standard t-tests / z-tests (rely on CLT)
- **Small samples/complex metrics:** Use bootstrapping
- **Extreme outliers:** Robust statistics (better models!)

# Simple analysis 👩‍💻 (1,500 users/group, 5 pp lift)

```python
from experiment_utils import ExperimentAnalyzer

analyzer_simple = ExperimentAnalyzer(
    df,
    treatment_col='treatment',
    outcomes='conversion',
    bootstrap=True,
    exp_sample_ratio=0.50,
    outcome_models={'conversion':['ols', 'logistic']},
    # correction='sidak',
)
analyzer_simple.get_effects()
print(analyzer_simple.results.round(3)[['model_type', 'absolute_effect', 'relative_effect', '
```

```
  model_type  absolute_effect  relative_effect  srm_pvalue
0        ols            0.051            0.153         1.0
1   logistic            0.051            0.153         1.0
```

# Regression with Pre-Treatment Covariates 🤖

**Regression adjustment** controls for pre-treatment covariates to partial out noise unrelated to treatment.

**Benefits (even with perfect randomization):**

- Reduced variance $\rightarrow$ narrower confidence intervals, higher precision
- Increased power (especially when covariates correlate with outcome)
- Corrects chance imbalances in small samples ($n < 1000$/group)
- **Use when:** pre-treatment covariates available — almost always worth it

# Regression Adjustment 🤖

```python
from experiment_utils import ExperimentAnalyzer

# no adjustment
analyzer_simple = ExperimentAnalyzer(
    df,
    treatment_col='treatment',
    outcomes='conversion',

)
analyzer_simple.get_effects()
print(analyzer_simple.results[['absolute_effect', 'pvalue', 'standard_error']])
```

```
   absolute_effect    pvalue  standard_error
0         0.051333  0.003411        0.017531
```

```python
# covariate adjustment
analyzer_adjusted = ExperimentAnalyzer(
    df,
    treatment_col='treatment',
    outcomes='conversion',
    regression_covariates=['age', 'previous_purchases', 'days_since_signup']
)

analyzer_adjusted.get_effects()
print(analyzer_adjusted.results[['absolute_effect', 'pvalue', 'standard_error']])
```

```
   absolute_effect    pvalue  standard_error
0         0.050259  0.000039        0.012212
```

```python
final_effect = analyzer_adjusted.results.loc[0, 'absolute_effect']
```

# Winner's Curse 💀 (Subsample of 500 users/group)

**Problem:** Significant results often overestimate the true effect

**Why?**

- Selection bias: Only "winners" reported
- Small samples + low power → greater exaggeration (2-3x for power < 0.50)

$$\text{Exaggeration (Type M)} = \left|\frac{\hat{\tau}}{\tau}\right|$$

$$\text{Relative Bias} = \frac{\hat{\tau}}{\tau}$$

When an underpowered experiment reports a significant effect, divide the estimate by the exaggeration ratio to get a more realistic sense of the true effect size.

# Winner's Curse 💀

```
# we hunted for significant effect using a smaller sample (n=500/group)
print(analyzer_retro.results[['absolute_effect', 'pvalue', 'standard_error']])
   absolute_effect    pvalue  standard_error
0         0.080322  0.007778        0.030179

print(f'True effect: {final_effect:.4f}')
True effect: 0.0503

cols = ['power', 'type_s_error', 'type_m_error', 'relative_bias', 'trimmed_abs_effect']
print(analyzer_retro.calculate_retrodesign(true_effect=final_effect)[cols])
    power  type_s_error  type_m_error  relative_bias  trimmed_abs_effect
0  0.3996           0.0        1.5829         1.5829            0.050743
```

# Non-Compliance & Confounding

# What If the Experiment Is Broken? 😱

Common problems:

- **Implementation issues** - Bugs in assignment or logging
- **SRM (Sample Ratio Mismatch)** - Observed split differs from expected
- **Non-compliance** - Users don't receive assigned treatment

What can we do?

- **ITT (Intent to Treat)** - Preserves randomization, underestimates effect
- **Regression adjustment / IPW (Inverse Probability Weighting)** - Correct for imbalances
- **IV (Instrumental Variables)** - Assignment as instrument (ITT $\rightarrow$ LATE)

# Simulation: Non-Compliance + Confounding 👩‍💻

**CS farming meeting** scenario:

- Only a % of treated users attend (one-sided non-compliance)
- Attendees are more engaged, higher baseline revenue (confounding)

**How to recover the true effect on bookings?**

| | |
|---|---|
| **ITT** | As assigned → underestimates effect |
| **Regression** | Covariate adjustment → reduces bias |
| **IPW** | Inverse probability weighting → corrects imbalance |
| **IV** | Assignment as instrument → LATE ≡ ATT (one-sided non-compliance) |

# Naive Approach 🧸

**Naive analysis:** Compare attendees vs non-attendees

**True effect** = 5 bookings

```
naive = ExperimentAnalyzer(
    cdf,
    treatment_col='attended',
    outcomes='bookings',
)

naive.get_effects()

naive.results[['absolute_effect', 'abs_effect_lower', 'abs_effect_upper', 'pvalue']]
```

```
absolute_effect  abs_effect_lower  abs_effect_upper         pvalue
0         6.316905          5.921429          6.712381  3.823925e-215
```

# Intent-to-Treat (ITT) ⚖️

**ITT:** Compare all treated vs control, regardless of attendance

**True effect** = 5 bookings

```
itt = ExperimentAnalyzer(
    cdf,
    treatment_col='assigned',
    outcomes='bookings',
)
itt.get_effects()

itt.results[['absolute_effect', 'abs_effect_lower', 'abs_effect_upper', 'pvalue']]
```

```
absolute_effect  abs_effect_lower  abs_effect_upper        pvalue
0         2.743512          2.359612          3.127413  1.418363e-44
```

# Inverse Probability Weighting (IPW) 🥷

**IPW:** Adjust for compliance selection using covariates

**True effect** = 5 bookings

```
ipw = ExperimentAnalyzer(
    cdf,
    treatment_col='attended', outcomes='bookings',
    balance_covariates=['engagement', 'account_age_months', 'monthly_usage'],
    adjustment='balance',  balance_method='ps-logistic', estimand='ATT', overlap_plot=True)

ipw.get_effects()
ipw.results[['absolute_effect', 'abs_effect_lower', 'abs_effect_upper', 'pvalue']]
   absolute_effect  abs_effect_lower  abs_effect_upper         pvalue
0         5.056775          4.652374          5.461177  1.212354e-132
```

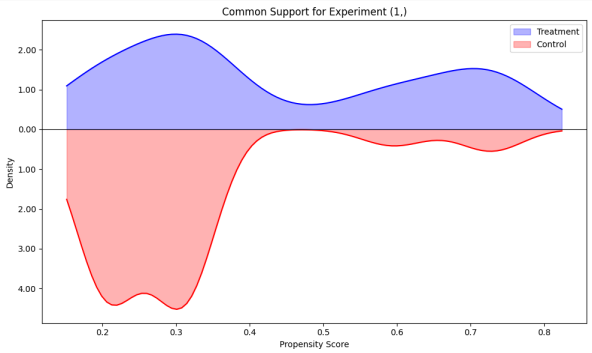
# Overlap Assumption 🔍

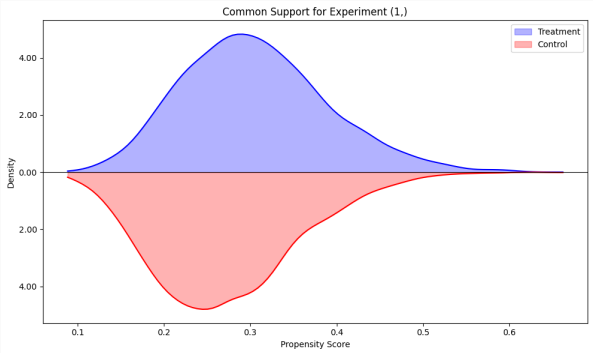
## Why it matters:

- IPW requires common support: every unit needs a non-zero probability of either treatment
- Without it, weights become extreme $\rightarrow$ high variance and bias

## What to do:

- **Diagnose:** Inspect propensity score distributions by group
- **Trim:** Drop or down-weight units outside common support
- **Target:** Shift estimand from ATE $\rightarrow$ ATT



# Instrumental Variables (IV) 🔑

**IV:** Use assignment as instrument to isolate the effect on compliers (attendees)

One-sided non-compliance: LATE $\equiv$ ATT, since there are no always-takers.

**True effect** = 5 bookings

```
iv = ExperimentAnalyzer(
    cdf,
    treatment_col='attended',
    outcomes='bookings',
    regression_covariates=['engagement', 'account_age_months', 'monthly_usage'],
    instrument_col='assigned',
    adjustment='IV'
)

iv.get_effects()

print(iv.results[['absolute_effect', 'abs_effect_lower', 'abs_effect_upper', 'pvalue']])
   absolute_effect  abs_effect_lower  abs_effect_upper  pvalue
0         4.870595          4.381111           5.36008     0.0
```

# Fixed-Effects Meta-Analysis 📊

**Goal:** Pool effect estimates across multiple experiments into a single estimate

**Inverse-variance weighting:**

- Each experiment's estimate is weighted by $w_i = 1/\sigma_i^2$
- More precise experiments (smaller SE) get more weight

**When to use it:**

- Multiple experiments testing the same intervention
- Any single experiment underpowered on its own
- Experiments run across different regions / segments

**Key assumption:** All experiments share the same true effect (homogeneity)

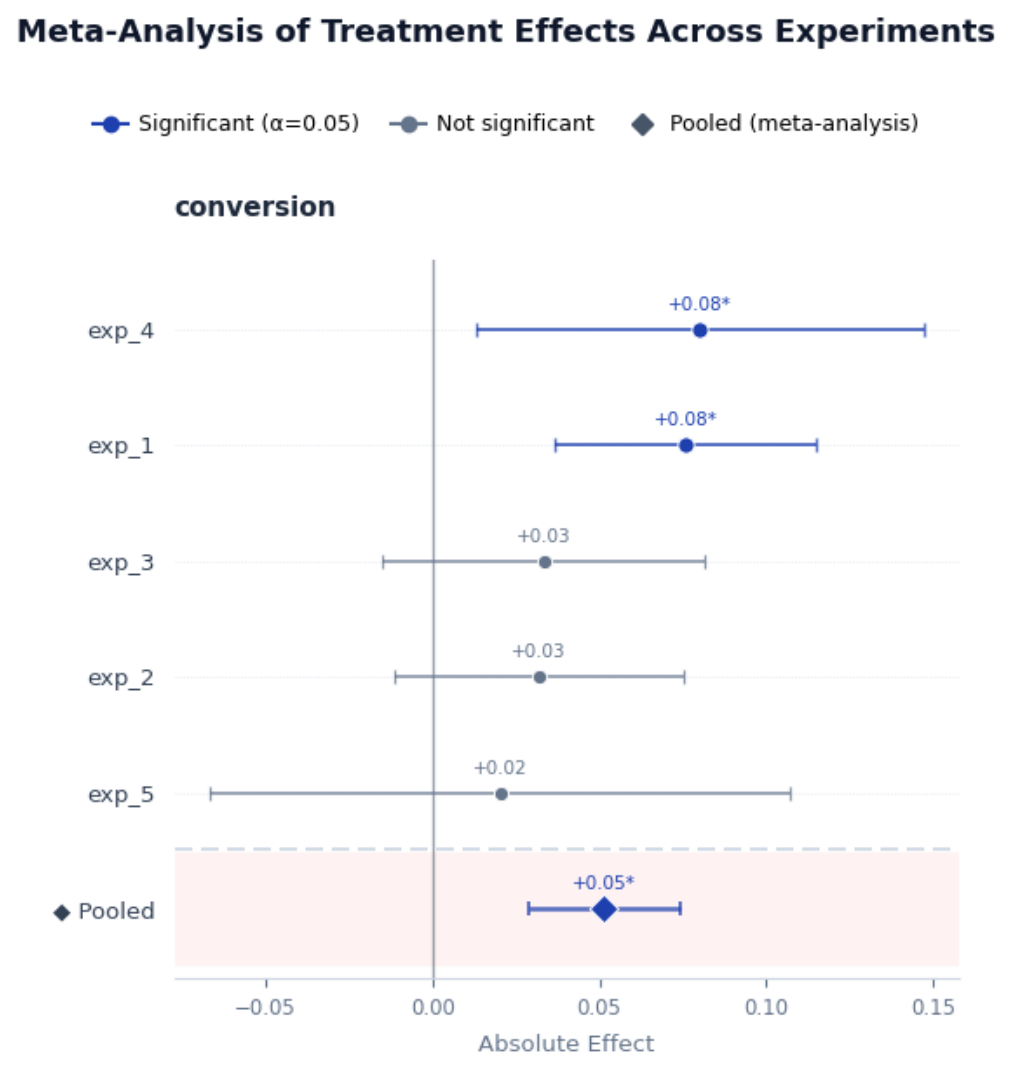
# Fixed-Effects Meta-Analysis 👩‍💻

**Setup:** 5 experiments, baselines correlated with allocation → naive pooling is biased

```
# high-alloc -> low baseline; low-alloc -> high baseline
experiments = [
    {"name": "exp_1", "n": 3000, "alloc": 0.20, "baseline": 0.20},
    {"name": "exp_2", "n": 2000, "alloc": 0.30, "baseline": 0.25},
    {"name": "exp_3", "n": 1500, "alloc": 0.50, "baseline": 0.30},
    {"name": "exp_4", "n": 1000, "alloc": 0.70, "baseline": 0.40},
    {"name": "exp_5", "n": 800,  "alloc": 0.80, "baseline": 0.50},
]

analyzer = ExperimentAnalyzer(
    data=meta_df, treatment_col="treatment",
    outcomes=["conversion"], experiment_identifier="experiment",
)
analyzer.get_effects()

pooled = analyzer.combine_effects(grouping_cols=["outcome"])
print(pooled[["outcome", "experiments", "absolute_effect", "standard_error", "pvalue"]])
      outcome  experiments  absolute_effect  standard_error    pvalue
0  conversion          5.0          0.05144        0.011513  0.000008

# True effect = 0.05
# Naive pooled  = 0.14  <-- 2.9x overestimate
# Meta-analysis = 0.05  <-- recovers true effect
```

# Fixed-Effects Meta-Analysis 👩‍💻

```
fig = analyzer.plot_effects(meta_analysis=True)
```



# Key takeaways ✨

- Power analysis is crucial for experiment design; MDE should reflect business relevance, not statistical convenience
- Sampling methods (random vs stratified) can improve balance and precision
- Regression adjustment reduces variance and corrects for imbalances, even with perfect randomization
- Non-compliance and confounding can be addressed with ITT, IPW, and IV methods
- Meta-analysis allows pooling across multiple experiments

# Links

- Simulation code
- `experiment-utils-pd`